

SUMMARY

RAG evidence selection: coverage 66.25% → 91.25%, false evidence 20 → 0

Agent execution environment: user-defined skills behind approval, edit quality 82.5% → 91.2% (114 drafts)

Data pipelines: PDF conversion pass rate 45.17% → 89.89%, drug conversion \$200 estimated → \$11.18 spent

PROJECTS & EXPERIENCE

Fruition

2026.04 — Present

LLM Wiki-based AI workspace · AI SW Maestro, 17th cohort · Deployed on AWS EKS, in QA
Team project (3 + designer) / AI lead

How it started: a teammate's pain point with managing documents, refined through design thinking — saved documents could not be found when needed.
Team: me (all AI), 1 backend (Spring backend), 1 frontend & DevOps (frontend, MSA deployment on AWS), external designer · 3 mentors

[GitHub · AI repository ↗](#)

Improving retrieval for grounded AI answers

Finds the document evidence AI answers need, and attaches none to questions the documents cannot answer.

- Evidence selection for RAG

Problem: Retrieval missed required facts and returned evidence for unanswerable questions.

Solution: Applied Jev, a model that picks only from a fixed set of options, after routing tests, then controlled candidates and return limits after a full-corpus trial. Tuned to 300 candidates and up to 4 parallel batches.

Decision: GPT selection was too costly and slow; tuning weights and RRF gave similar scores across settings, so I adopted Jev.

Result: Across 9 papers, coverage on 80 answerable questions rose from 66.25% to 91.25%. False evidence returns on 20 unanswerable questions fell from 20 to 0. Median time before/after Jev tuning: 9.090s → 2.251s (still slower than the original 0.431s).

Agent execution environment - skills, approval, routing and conversation context

Designed reusable Skills with user approval before changes. Applied Jev to request routing, and set the scope of the conversation context and edit input the model receives.

- User-defined task instructions (Skills)

Problem: Developers cannot build every recurring workflow users need.

Solution: Designed drafting, review and publishing of natural-language instructions, with checks against bypassing permissions or approval, implemented with AI coding tools. Document changes require separate preview and approval.

Result: Published Skills became available to the Agent. In a 6-document check, no changes occurred before approval; 3 folders were created and documents moved afterward.

- Jev-based request routing

Problem: Misclassifying questions as edits led to the wrong action.

Solution: Applied Jev to select task types and additional conditions for questions, edits, document creation and folder organization.

Decision: As a newly released model, validated it first on low-risk request routing.

Result: Fully correct decisions on the same 98 requests rose from 77 to 81 (63 for GPT-5-nano on the same options). Median decision time fell from 8.967s to 0.658s.

- Conversation memory

Problem: a long conversation cannot be passed to the model in full.

Solution: keep the last six messages verbatim, compress older ones into a rolling summary, and leave the summary unchanged when summarization fails. The router also receives earlier routing decisions, so referring expressions resolve against earlier turns.

- Bounding the edit input

Problem: sending a whole document to the model drives up cost and latency.

Solution: send only the target line range plus surrounding context, and reject ranges that straddle a table or code-block boundary. A benchmark tracks input size against the source and average and p95 time.

Execution reliability and LLM tracing

Undoing changes an AI task already saved, and making each stage of a run inspectable. Execution spans several stores and cannot be one transaction, so recovery runs as reverse actions.

- Undoing a cancelled task in reverse

Problem: an AI task saves documents, folders, wiki pages and skills between model calls, so stopping execution leaves some changes behind.

Solution: persist the cancellation first to block later writes, then undo children first using a pre-change snapshot and an idempotency key tied to the approval. Recovery uses the run record alone, never a model call, and cancellation is final only once full recovery is confirmed.

- Recovering without erasing other tasks' contributions
Problem: a concept page merges contributions from several documents, so an older snapshot would erase valid contributions from other documents.
Solution: record operations, revisions and per-document contributions append-only, then reassemble with only the target contribution excluded. Soft delete instead of removal keeps the audit trail.
- LLM run tracing and the boundary around sensitive data
Problem: the evaluation loop ran as plain Python, so stages could not be inspected separately, and whatever reached the tracing tool could carry sensitive data.
Solution: rebuilt the loop as a LangGraph graph so generation, evaluation and retry each trace as a node. The agent graph keeps tracing off and excludes raw tool lookups from persisted state, and my own prompt log masks numeric personal data before writing. Timing runs disable tracing and logging entirely.

Document conversion pipeline and faster Wiki generation

Converted uploaded PDFs for retrieval and grounded answers, and reduced Wiki generation time.

- PDF → Markdown conversion
Problem: Open-source conversion damaged tables and equations; full AI conversion still had errors and high API usage.
Solution: Compared PDF tools and adopted AnyDoc. Repaired damaged text with AI, processed tables and equations separately, and preserved original figures.
Decision: Used fast AnyDoc for body text, and moved repair from an sLLM to commercial APIs to avoid GPU server costs.
Result: Internal pass rate on 30 PDF pages improved from 45.17% to 89.89%.
- Parallel Wiki generation
Problem: Later documents waited for earlier external AI responses.
Solution: Compared worker counts and requests per worker. Capped workers at 4 to manage memory; rejected extra concurrency and GPU parallelism that slowed processing.
Result: Analysis of 4 documents fell from 282.11s to 73.89s (about 74% less).

MSA: Splitting only by feature would keep DB load the same while adding hops, so I proposed splitting by data ownership and co-designed boundaries and communication with teammates and mentors.

Internal evaluations. RAG timing compares Jev before/after tuning. Parallel document timing covers analysis only, excluding storage and post-processing.

See the detailed portfolio for the problem-solving process and experimental results.

[RAG evidence selection ↗](#) [Agent execution environment ↗](#) [PDF conversion ↗](#) [Parallel document processing ↗](#)

Pilltip

2025.03 — 2025.12

Personalized AI medication service · Pusan National University capstone · 6 users on internal APK builds
 Team project (3 + designer) / Core backend · data preparation

How it started: drug information is scattered, making it hard to check whether a medicine is safe for your current health condition.

Team: me (backend & AI), 1 Android (Kotlin app), 1 backend & security (user DB, login, questionnaires, AES-GCM), designer. The features below are my work.

[GitHub repository ↗](#)

Drug data preparation pipeline

Rewrote about 44,000 drug records in plain language, converting each repeated sentence once and reusing it per drug.

- Problem:** Converting all 44,000 drug records was estimated to cost about \$200 in API calls.
Solution: Split text into sentence blocks, merged exact duplicates, converted via GPT Batch API and mapped results back to each drug. Reproduced in PySpark in 2026 to measure ending normalization and MinHashLSH.
Decision: The job needed no real-time response, so Batch API traded waiting time for lower cost.
Result: Actual API spending was \$11.18 versus about \$200 estimated for all original text. In the reproduction, exact deduplication cut text from 744MB to 44.1MB (−94.1%).

Estimated cost versus actual spending; preprocessing labor is excluded.

AI medication chatbot using health profiles

Found drug and medication information from natural-language symptoms. Evaluated health profiles internally, separately from information sent to external AI.

- Problem:** Everyday symptom language differed from drug indications, while sensitive health data needed to stay out of external AI inputs.
Solution: Used Weaviate semantic search and AI orchestration for drug discovery, medication risks and dosage. Evaluated pregnancy, current medications and underlying conditions in internal code.
Decision: Filtering the profile before sending had too many cases to cover, so the fields were excluded and judged internally.
Result: Implemented personalized symptom-based search while excluding those profile fields from external LLM inputs.

Backend feature implementation

Built server features for drug search, risk information, medication reminders and logs, and family profiles. Integrated with teammates responsible for the mobile app, users and security.

- Designed the drug database on Java and Spring Boot with search, autocomplete and DUR risk display. Cut search responses from 5–7s to 0.8s on average with Elasticsearch and Redis.
- Built FCM medication reminders and logs, deep-link friend invites and family profile switching, and kept API docs, meeting notes and ADRs in Notion to share decisions.

Feature work driven by feedback from live exhibitions

We ran the booth twice as Pusan National University's representative exhibition team, demonstrating to more than 100 visitors and collecting 10 pieces of feedback. Most asked for health supplements, so I built that within the data we already had.

- Extending support to health functional foods
Problem: at the first exhibition visitors could only check prescription drugs and asked to see health functional foods too.
Solution: within the data we held, added them to search and autocomplete, surfaced caution and co-administration information, allowed them in reminders and logs, and included them in chatbot answers.
Result: a request from the floor became the next slice of work. Limited to the data we had obtained, not the exhaustive coverage we have for drugs.
- Feedback I could not act on
Someone pointed out that the service would be better off running in containers, but I could not adopt it inside the capstone timeline. On Fruition I designed the AI services around a container-based deployment from the start.

See the detailed portfolio for the problem-solving process and experimental results.

[Data pipeline case study ↗](#) [Personalization case study ↗](#)

Skills

AI & Search LangChain · LangGraph · Spring AI · RAG · LLM Evaluation · Prompt injection defense
LangSmith · Elasticsearch · Weaviate

Backend Java · Spring Boot · Python

Data & Infra MySQL · PostgreSQL · Redis · Kafka
Docker · Docker Compose · GitHub Actions

Activities & contributions

2026.07 — 2026.09

200 SIGNAL · TQQQ 200-day moving average signal app (personal project)

I turned a publicly posted TQQQ 200-day moving average strategy into a written spec and an Android app on live quotes, used by 2 family members. I marked the source post's claimed returns as unverified and reproduced 45 cycles, measuring a 40.00% win rate rather than the claimed 19%. Counting partial exits as trades lifts that to 72.73%, showing the post's two win rates count different units. I backtested 5 strategy variants under the same assumptions, comparing CAGR and MDD across sub-periods. At the users' request I added a second QQQ-based strategy and SGOV/SPYM handling for cash waiting on an entry.

The app was never released, and account data stays on the device.

[GitHub repository ↗](#)

2026.06 — 2026.06

Rhwp open-source contributions

Gave Claude Code a work plan and contribution rules to automate the flow from issue analysis to PR, including HWP/HWPX save-error fixes; 7 PRs merged and listed as a contributor.

[GitHub repository ↗](#) [PR #1213 · Merged ↗](#) [PR #1351 · Submitted ↗](#)

2025.03 — 2026.01



APPTIVE, Pusan National University

Backend mentee and mentor. Led 6 mentoring and code-review sessions for 12 mentees. Helped revise the curriculum to strengthen HTTP, Servlet, REST API and database foundations.

APPTIVE Backend Mentor Merit Award · 2026.01.09

2025.11 / 2025.07



SK AI SUMMIT · K-ICT WEEK in Busan

Represented Pusan National University at the Pilltip booth, demonstrating the service to more than 100 visitors and answering technical questions. Most of the 10 pieces of feedback asked for health supplement coverage.

2022.02 — 2025.07

945-day Baekjoon problem-solving streak

Longest streak of 945 consecutive days on solved.ac. 1,659 problems solved, Platinum IV.

[Baekjoon solutions repository ↗](#)

Education & certification

2022.02 — 2026.02
(Transfer)

Pusan National University

GPA 4.12 / 4.5

2019.03 — 2022.02
(Withdrew)

Daegu University

GPA 4.2 / 4.5

2025.09

Engineer Information Processing

Human Resources Development Service of Korea

Awards

2025.10



Capstone Design Gold Award
Software & AI division / College of Information and Biomedical Engineering, Pusan National University

2025.09



Busan DATA WEEK Top Excellence Award
Data utilization case competition / Busan Techno Park

2025.09



AI LAUNCH Career School Startup Hackathon Encouragement Award
KRYPTON X x Root Impact x Google.org

2025.08



SW University Digital Competition Sponsor Award
SW University Council